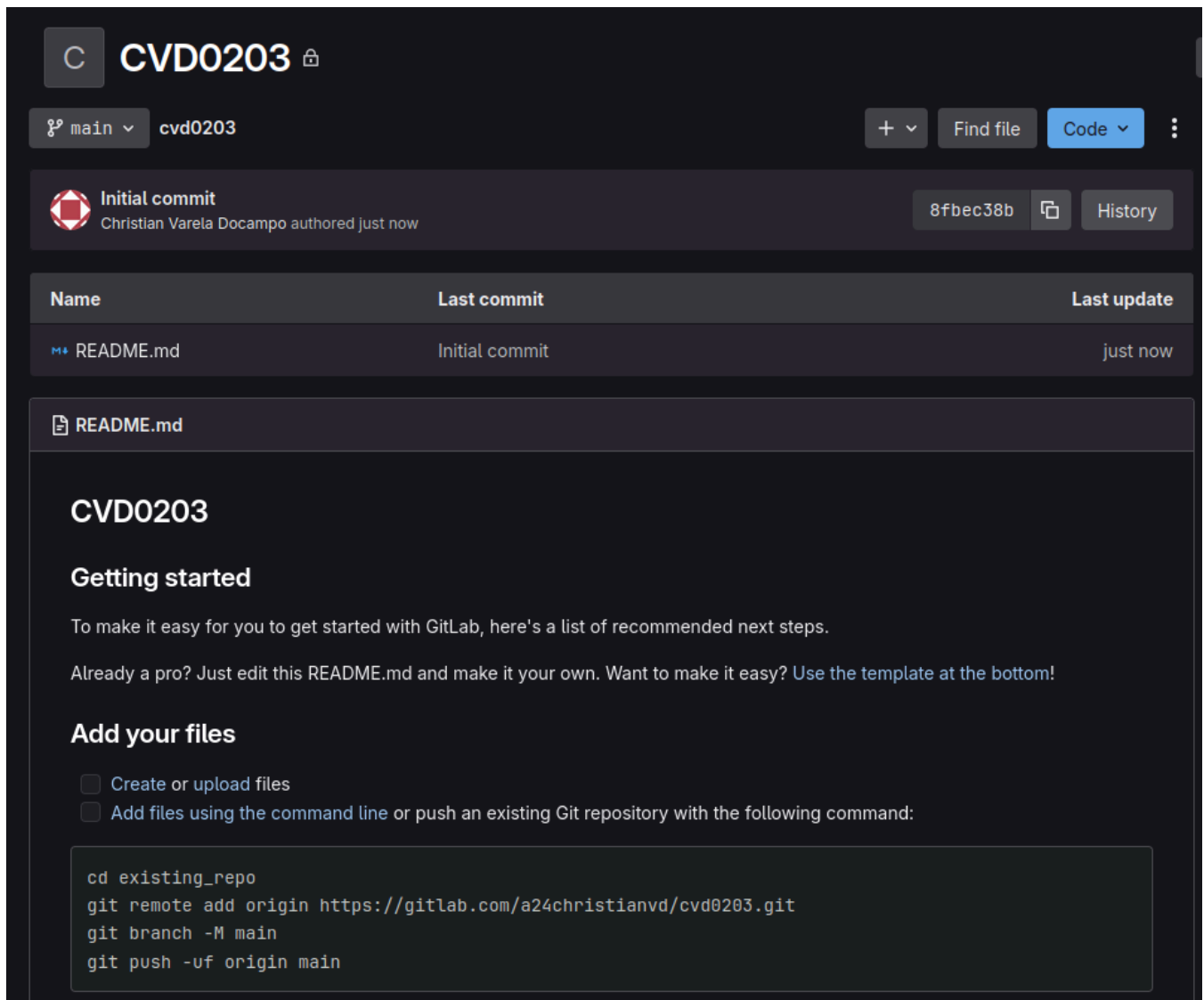


Exercicio 1: Probas automatizadas mediante CI/CD

Explicación Comezaremos o exercicio creando un repositorio ne GitLab e engadindo o código da nosa aplicación.

1. Crea un novo proxecto (en GitLab chámanlle proxectos ao repositorios) de nome `iniciaist0203` con ficheiro `Readme.md`. **Realiza unha captura** da interface web co proxecto en GitLab.



Clona o repositorio.

Dentro do repositorio crea o seguinte ficheiro `dni_validator.py` con este contido:

```
import re

def validar_dni(dni):
    return True
```

Crea tamén o ficheiro `test_dni_validator.py` co seguinte contido:

```
import unittest
from dni_validator import validar_dni
```

```

class TestDniValidator(unittest.TestCase):

    def test_dni_correctos(self):
        self.assertTrue(validar_dni("12345678Z")) # Exemplo válido
        self.assertTrue(validar_dni("00000000T"))
        self.assertTrue(validar_dni("98765432M"))
        self.assertTrue(validar_dni("99999999R"))

    def test_dni_incorrectos(self):
        self.assertFalse(validar_dni("12345678A")) # Letra incorrecta
        self.assertFalse(validar_dni("1234567Z")) # Faltan números
        self.assertFalse(validar_dni("123456789Z")) # Sobran números
        self.assertFalse(validar_dni("ABCDEFGHYZ")) # Letras no número
        self.assertFalse(validar_dni("12345678")) # Sen letra
        self.assertFalse(validar_dni("")) # Baleiro

if __name__ == '__main__':
    unittest.main()

```

1. Sube o código ao repositorio en GitLab.com.

Explicación O noso repositorio conta con dous ficheiros:

- `dni_validator.py`

: contén unha función que valida o DNI (de momento sen implantar)

```
test_dni_validator.py
```

- : conta cuns test unitario para comprobar o correcto funcionamento desa función.

De seguido utilizaremos **CI/CD de GitLab para realizar o test automaticamente** en canto se suba unha nova versión a GitLab.com

1. Crea o ficheiro `.gitlab-ci.yml`. Neste ficheiro define unha única `stage` de nome `probas`. Crea un novo `job` para a `stage` anterior de nome `unitarias`. Neste `job` debes de:

- Utiliza a imaxe: `python:3.9`
- Executar os seguintes comandos (O segundo destes comandos é o que executa as probas):

```

echo "Comezando os test..."
python -m unittest test_dni_validator.py

```

Fai un `commit` e un `push`. **Entrega captura** do contido do ficheiro `.gitlab-ci.yml`

```
image.png dni_validador.py test_dni_validator.py .gitlab-ci.yml x
ud02 > 04 > 01 > cvd0203 > .gitlab-ci.yml
1 stages:
2 | - probas
3
4 unitarias:
5 | image: python:3.9
6 | stage: probas
7 | script:
8 |   - echo "Comezando os test..."
9 |   - python -m unittest test_dni_validator.py

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS QUERY RESULTS
a24christianvd@a01eql03:~/Documentos/Despregamento/ud02/04/01/cvd0203$ git status
.gitlab-ci.yml

nothing added to commit but untracked files present (use "git add" to track)
a24christianvd@a01eql03:~/Documentos/Despregamento/ud02/04/01/cvd0203$ git add .
a24christianvd@a01eql03:~/Documentos/Despregamento/ud02/04/01/cvd0203$ git commit -m "ci"
[main 09a1b70] ci
1 file changed, 9 insertions(+)
create mode 100644 .gitlab-ci.yml
a24christianvd@a01eql03:~/Documentos/Despregamento/ud02/04/01/cvd0203$ git push
Enumerating objects: 4, feito.
Counting objects: 100% (4/4), feito.
Delta compression using up to 16 threads
Compressing objects: 100% (3/3), feito.
Writing objects: 100% (3/3), 387 bytes | 387.00 KiB/s, feito.
Total 3 (delta 1), reused 0 (delta 0), pack-reused 0
To https://gitlab.com/a24christianvd/cvd0203.git
e1df67..09a1b70 main -> main
```

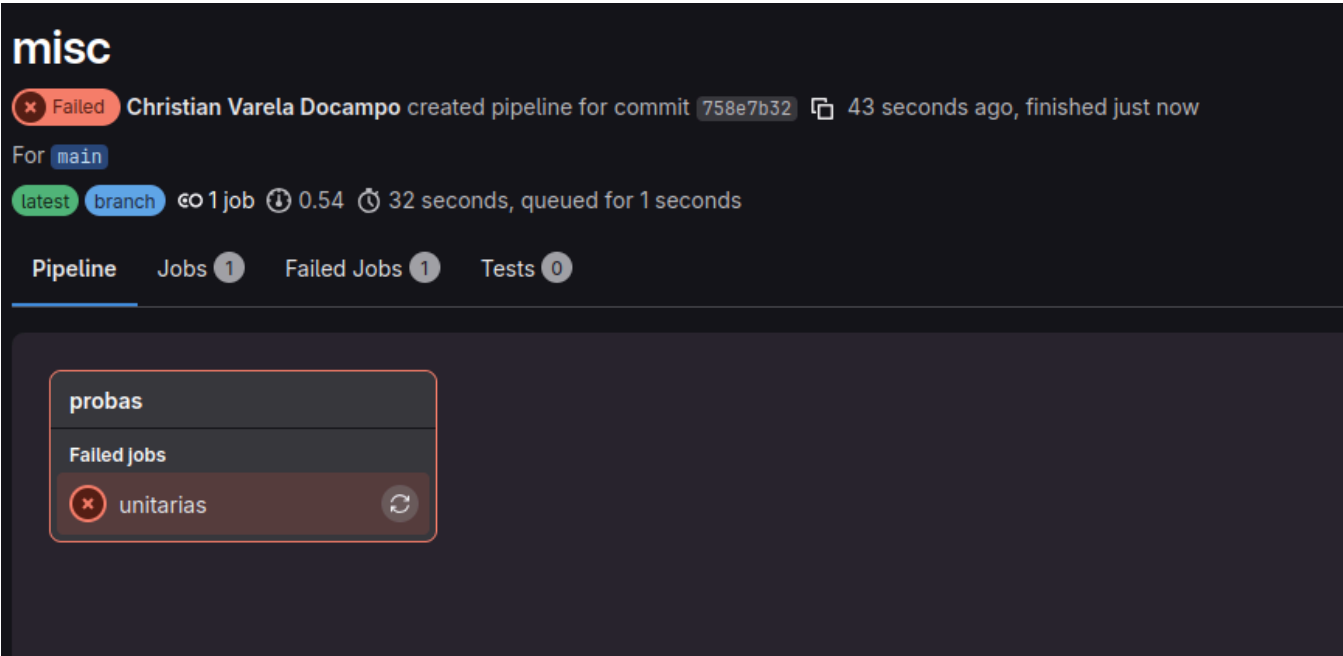
Explicación Unha vez subida a nova versión de repositorio, debería lanzarse automaticamente as probas en GitLab.com. **Vamos a ver os resultados destas probas na interface de GitLab.**

1. Abre o proxecto, e vai no menú a `Build > Pipelines`. **Realiza unha captura dos pipelines** lanzados.

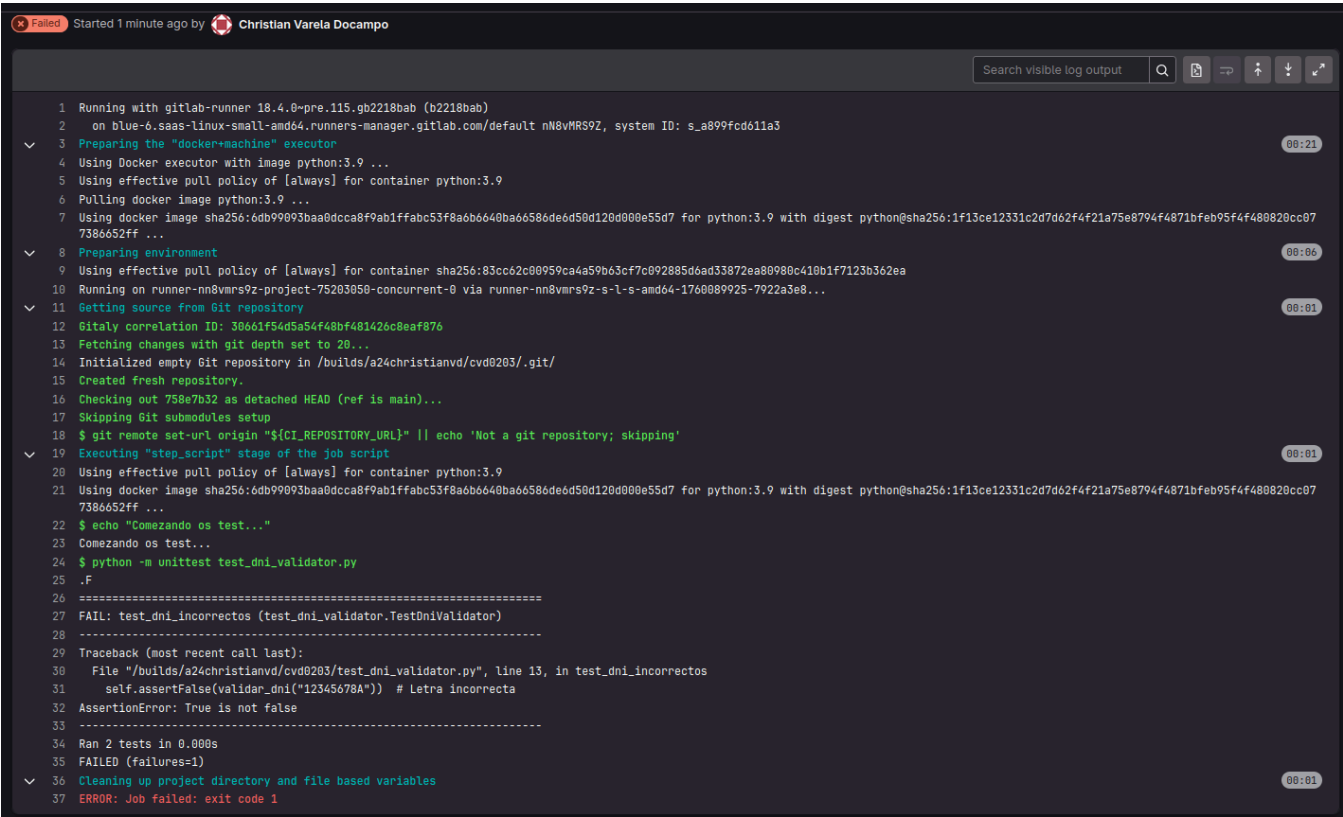
Status	Pipeline	Created by	Stages	Actions
 Failed 58 seconds ago	ci #2092359348  main   latest  error  branch			

Deberías ver que o estado é `Failed`. Preme no estado e **realiza unha captura** do contido que se mostra. Deberías ver un resumo do `pipeline` que se executou.

(O commit é diferente porque tiven que facer outro despois de verificar o gitlab)



Agora selecciona a lapela `jobs`. Na táboa que se mostra, selecciona o enlace da columna `job` para ver a execución deste. **Realiza unha captura** da execución.



No última captura podemos observar que a nosa función non pasou os test unitario. Lóxico xa que non está implantada. Agora poderíamos ir implantando a nosa función e facendo `push` ata que a nosa función estea correctamente implantada. Vamos xa ver como sería o resultado correcto.

Modifica o ficheiro `dni_validator.py` con este contido:

```
import re

def validar_dni(dni):
    """
    Valida se un DNI español é correcto en formato e letra.

    Args:
        dni (str): DNI a validar (ex: '12345678Z')

    Returns:
        bool: True se é válido, False se non.
    """
    dni = dni.upper()
    patron = r'^\d{8}[A-Z]$'

    if not re.match(patron, dni):
        return False

    letras = "TRWAGMYFPDXBNJZSQVHLCKE"
    numero = int(dni[:8])
    letra_correcta = letras[numero % 23]

    return dni[-1] == letra_correcta
```

Fai un `commit` e un `push`. Abre o proxecto, e vai no menú a `Build > Pipelines`. **Realiza unha captura** dos `pipelines` lanzados.

Status	Pipeline	Created by	Stages	Actions
✓ Passed 00:00:32 just now	editado #2092392069 main d48aa3a9 latest branch		✓	

Deberías ver que o estado é `Passed`. Preme no estado e **realiza unha captura** do contido que se mostra. Deberías ver un resumo do `pipeline` que se executou.

editado

✓ Passed

Christian Varela Docampo created pipeline for commit d48aa3a9 59 seconds ago, finished 25 seconds ago

For main

latest

branch

1 job

0.53

32 seconds, queued for 1 seconds

Pipeline

Jobs 1

Tests 0

probas

✓ unitarias

1. Agora selecciona a lapela `jobs`. Na táboa que se mostra, selecciona o enlace da columna `job` para ver a execución deste. **Realiza unha captura** da execución.

unitarias

Passed Started 1 minute ago by Christian Varela Docampo

Search visible log output

```
1 Running with gitlab-runner 18.4.0-pre.115.gb2218bab (b2218bab)
2   on blue-3.saas-linux-small-amd64.runners-manager.gitlab.com/default zxwgkjAPH, system ID: s_d5d3abdbf08a
3   ✓ Preparing the "docker+machine" executor
4   Using Docker executor with image python:3.9 ...
5   Using effective pull policy of [always] for container python:3.9
6   Pulling docker image python:3.9 ...
7   Using docker image sha256:6db99093baa0dcca8f9ab1ffabc53f8a6b6640ba66586de6d50d120d000e55d7 for python:3.9 with digest python@sha256:1f13ce12331c2d7d62f4f21a75e8794f4871bfeb95f4f480820cc077386652ff ...
8   ✓ Preparing environment
9   Using effective pull policy of [always] for container sha256:dc96c9609b410d5ca3bc6525711d63100224e951914484d08890cb7adacfb1f5
10  Running on runner-zxwgkjaph-project-75203050-concurrent-0 via runner-zxwgkjaph-s-l-s-amd64-176009063-de5107f6...
11  ✓ Getting source from Git repository
12  Gitly correlation ID: d57a082ff81044dc99924ddbfa8598e2
13  Fetching changes with git depth set to 20...
14  Initialized empty Git repository in /builds/a24christianvd/cvd0203/.git/
15  Created fresh repository.
16  Checking out d48aa3a9 as detached HEAD (ref is main)...
17  Skipping Git submodules setup
18  $ git remote set-url origin "${CI_REPOSITORY_URL}" || echo 'Not a git repository; skipping'
19  ✓ Executing "step_script" stage of the job script
20  Using effective pull policy of [always] for container python:3.9
21  Using docker image sha256:6db99093baa0dcca8f9ab1ffabc53f8a6b6640ba66586de6d50d120d000e55d7 for python:3.9 with digest python@sha256:1f13ce12331c2d7d62f4f21a75e8794f4871bfeb95f4f480820cc077386652ff ...
22  $ echo "Comezando os test..."
23  Comezando os test...
24  $ python -m unittest test_dni_validator.py
25  ..
26  .....
27  Ran 2 tests in 0.000s
28  OK
29  ✓ Cleaning up project directory and file based variables
30  Job succeeded
```

Exercicio 2: Build automatizado mediante CI/CD

Explicación Neste momento sabemos que a nosa función funciona correctamente porque pasa tódalas probas unitarias. Poderíamos utilizar esta función nun programa principal, no que se recibira o dni por argumento e se indicara se é correcto ou non.

1. Crea no repositorio o ficheiro `main.py` co seguinte contido:

```
import sys
from dni_validator import validar_dni

def main():
    if len(sys.argv) != 2:
        print("Uso: python main.py <DNI>")
        sys.exit(1)

    dni = sys.argv[1]
    if validar_dni(dni):
        print(f"O DNI '{dni}' é válido.")
    else:
        print(f"O DNI '{dni}' NON é válido.")

if __name__ == "__main__":
    main()
```

Executa o seguinte comando para ver se funciona correctamente o programa: `python3 main.py <o teu DNI>`. Realiza unha captura da execución do comando.

```
• a24christianvd@a01eql03:~/Documentos/Despregamento/ud02/04/cvd0203$ python3 main.py 46092191F
O DNI '46092191F' é válido.
```

1. Sube o repositorio ao GitLab.

Explicación Agora imos **automatizar o empaquetado do noso programa**, pero só se realizará se pasa as probas correctamente. Para iso engadiremos unha nova `stage`.

1. Engade unha nova `stage` que se execute só se o resultado das probas é satisfactorio. O nome da `stage` será `build`.

2. Crea un novo `job` para a `stage` anterior de nome `empaquetado`. Neste `job` debes de:

- Utiliza a imaxe: `python:3.9`
- Executar os seguintes comandos (O segundo destes comandos é o que executa as probas):

```
pip install pyinstaller # Instala o paquete que nos permite empaquetar o paquete
pyinstaller --onefile main.py # Realiza o empaquetado. O executable crease no
directorio dist co nome main
```

- Engade o seguinte dentro do `job`. Isto permitiranos acceder ao executable dende a interface e poder descargalo.

```
artifacts:
  paths:
    - dist/main
  expire_in: 1 week
```

Sube o repositorio a GitLab. **Entrega captura** do contido do ficheiro `.gitlab-ci.yml`.

```

ud02 > 04 > cvd0203 > 🔥 .gitlab-ci.yml
1  stages:
2    - probas
3    - build
4
5  unitarias:
6    image: python:3.9
7    stage: probas
8    script:
9      - echo "Comezando os test..."
10     - python -m unittest test_dni_validator.py
11
12 empaquetado:
13   image: python:3.9
14   stage: build
15   script:
16     - pip install pyinstaller
17     - pyinstaller --onefile main.py
18   dependencies:
19     - unitarias
20   artifacts:
21     paths:
22       - dist/main
23     expire_in: 1 week

```

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS QUERY RESULTS

```

a24christianvd@a01eq103:~/Documentos/Despregamento/ud02/04/cvd0203$ git add .
a24christianvd@a01eq103:~/Documentos/Despregamento/ud02/04/cvd0203$ git commit -m "ci"
[main a53d162] ci
1 file changed, 15 insertions(+), 1 deletion(-)
a24christianvd@a01eq103:~/Documentos/Despregamento/ud02/04/cvd0203$ git push
Enumerating objects: 5, feito.
Counting objects: 100% (5/5), feito.
Delta compression using up to 16 threads
Compressing objects: 100% (3/3), feito.
Writing objects: 100% (3/3), 461 bytes | 461.00 KiB/s, feito.
Total 3 (delta 1), reused 0 (delta 0), pack-reused 0
To https://gitlab.com/a24christianvd/cvd0203.git
eb2594f..a53d162  main -> main
a24christianvd@a01eq103:~/Documentos/Despregamento/ud02/04/cvd0203$ █

```

Abre o proxecto, e vai no menú a `Build > Pipelines`. Realiza unha captura dos `pipelines` lanzados. Deberías ver que o estado é `Passed`.

All 5 Finished Branches Tags View analytics Clear runner caches New pipeline				
Filter pipelines Q Show Pipeline ID				
Status	Pipeline	Created by	Stages	Actions
Passed 00:01:20 11 seconds ago	ci #2092408799 main a53d162b latest branch			Download

Na columna etapas, deberías ver que hai dous círculos en verde que identifican as dúas `stages` que temos. Preme no segundo e despois no `job` empaketado. **Realiza captura** do contido da web que se mostra.

```

Christian Varela Docampo / CVD0203 / Jobs / #11675083915
Search visible log output

61 327 INFO: Running Analysis Analysis-00.toc
62 327 INFO: Target bytecode optimization level: 0
63 328 INFO: Initializing module dependency graph...
64 329 INFO: Initializing module graph hook caches...
65 339 INFO: Analyzing modules for base_library.zip ...
66 949 INFO: Processing standard module hook 'hook-encodings.py' from '/usr/local/lib/python3.9/site-packages/PyInstaller/hooks'
67 1957 INFO: Processing standard module hook 'hook-pickle.py' from '/usr/local/lib/python3.9/site-packages/PyInstaller/hooks'
68 2788 INFO: Processing standard module hook 'hook-heapq.py' from '/usr/local/lib/python3.9/site-packages/PyInstaller/hooks'
69 3048 INFO: Caching module dependency graph...
70 3090 INFO: Analyzing /builds/a24christianvd/cvd0203/main.py
71 3092 INFO: Processing module hooks (post-graph stage)...
72 3102 INFO: Performing binary vs. data reclassification (1 entries)
73 3107 INFO: Looking for ctypes DLLs
74 3110 INFO: Analyzing run-time hooks ...
75 3115 INFO: Creating base_library.zip...
76 3135 INFO: Looking for dynamic libraries
77 3474 INFO: Warnings written to /builds/a24christianvd/cvd0203/build/main/warn-main.txt
78 3485 INFO: Graph cross-reference written to /builds/a24christianvd/cvd0203/build/main/xref-main.html
79 3495 INFO: checking PYZ
80 3495 INFO: Building PYZ because PYZ-00.toc is non existent
81 3496 INFO: Building PYZ (ZlibArchive) /builds/a24christianvd/cvd0203/build/main/PYZ-00.pyz
82 3577 INFO: Building PYZ (ZlibArchive) /builds/a24christianvd/cvd0203/build/main/PYZ-00.pyz completed successfully.
83 3587 INFO: checking PKG
84 3587 INFO: Building PKG because PKG-00.toc is non existent
85 3587 INFO: Building PKG (CArchive) main.pkg
86 9308 INFO: Building PKG (CArchive) main.pkg completed successfully.
87 9310 INFO: Bootloader /usr/local/lib/python3.9/site-packages/PyInstaller/bootloader/Linux-64bit-intel/run
88 9310 INFO: checking EXE
89 9310 INFO: Building EXE because EXE-00.toc is non existent
90 9310 INFO: Building EXE from EXE-00.toc
91 9310 INFO: Copying bootloader EXE to /builds/a24christianvd/cvd0203/dist/main
92 9311 INFO: Appending PKG archive to custom ELF section in EXE
93 9381 INFO: Building EXE from EXE-00.toc completed successfully.
94 9383 INFO: Build complete! The results are available in: /builds/a24christianvd/cvd0203/dist
95 Uploading artifacts for successful job
96 Uploading artifacts...
97 dist/main: found 1 matching artifact files and directories
98 Uploading artifacts as "archive" to coordinator... 201 Created correlation_id=0f8ba3beedf7402f9f182761f4457085 id=11675083915 responseStatus=201 Created token=6b_kFkcHs
99 Cleaning up project directory and file based variables
100 Job succeeded

```

Na parte dereita podes ver un apartado que pon `Artefactos e jobs`. Dende aí podes premer en `Descargar`. Preme e baixarase o noso programa empaketado. Descomprime o `zip`.

No terminal do equipo, vaite ao directorio onde descargaches o programa anterior. Executa o seguinte comando: `./main <o teu DNI>`. **Realiza captura** da execución deste comando.

```

a24christianvd@a01eql03:~/Documentos/Despregamento/ud02/04/02/dist$ ls
main
a24christianvd@a01eql03:~/Documentos/Despregamento/ud02/04/02/dist$ ./main 46092191F
0 DNI '46092191F' é válido.

```